Database Systems ISYS2014/ISYS5008

Student Name: Syed Mashood

Student ID: 21978586

Practical class: Wednesday 9:00AM

Introduction

This report covers how a relational database was built to handle info from global movie festivals. It looks at big events like Berlin's and Venice's, plus their prizes, movies up for awards or that won, along with directors, actors, and others tied in. The work walked through every step like starting with rough ER sketches, moving to structured planning, then setting it up in MySQL. Sample data from recent years filled the tables. Its use was shown by running practical SQL searches to pull useful insights. On top of that, smart database tools such as saved queries and automated routines got set up. Additionally, linking them to a Python program to show how it works outside theory. This effort shows how basic database ideas handle tough, everyday data tasks.

Design of the Database

Reasoning

It was the database that was built to properly model the complex and interconnected world of the film festival. The central actors in this field were used to determine the core entities:

Festival & Award: Each year, a festival (Festival) is a host to various Awards of various categories. This is a high degree of one-to-many relationship as without a festival, an award does not exist. Both Festival and Award allow the year attribute, which enables effective tracking of each edition of the festival.

Film and Person: A Film may contain several people (Person) including it in different capacities (e.g. director, actor, cinematographer) and a person may work on several films. The junction table of Film_Person resolves this many to many relationship. Person primary role attribute is used to define the most frequent profession of a person whereas role in film attribute of junction table is used to indicate his or her contribution to a specific film which is essential in offering the necessary flexibility.

Film & Nomination: A film may be nominated to more than one Award and an Award may have more than one film nominated. The Nomination table is the one that resolves this many to many relationship and is used as a central fact table. It does not only represent the relationship but also such important features as the result (Winner, Nominated, etc.), the nomination-date and notes that are optional.

Nomination & Person: It is a major innovation of this design to realize that a Nomination (e.g., of Best Actor) can be connected directly to a Person (or persons) who are the beneficiaries of that nomination. This is another many to many relationship, which is solved by Nomination_Person table. The design enables proper giving of accolades to individual persons, rather than the film itself.

Film & Genre: A Filmmay fall in several genres, and a Genre can be used to describe a large number of films. This many-to-many relationship is addressed by the Film Genre junction table which offers rich categorization.

Country: This table is an example of a normalized look up of Film production countries, Festival host countries and Person nationalities. This encourages consistency of data, minimizes redundancy as well as it is a characteristic feature of normalized design.

Data Types and Constraints

Textual information having the right length (e.g., title (400), name (300)) was stored in VARCHAR to strike a balance between storage and flexibility.

INT was applied to IDs, years and run times.

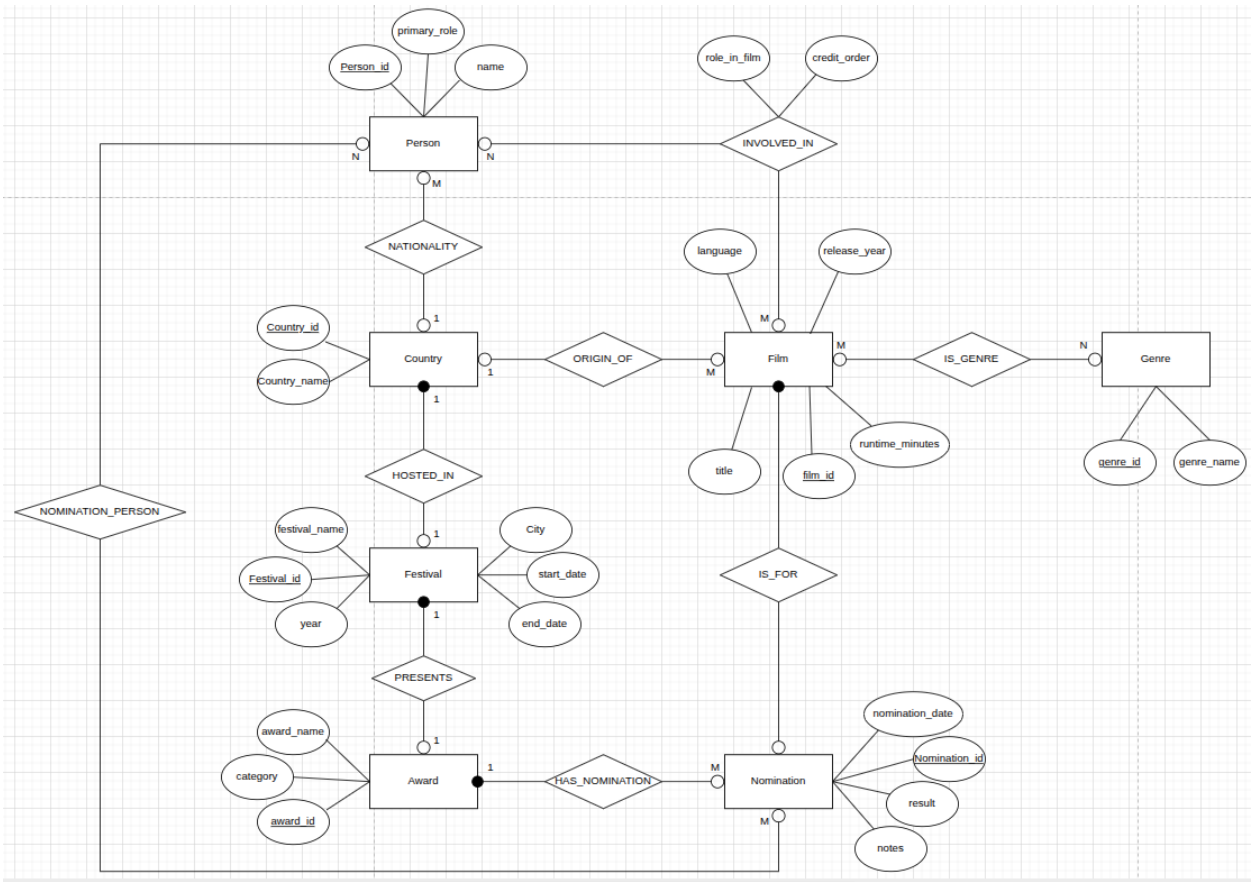
Start, end and nomination date were coded as DATE so that queries can be performed based on date.

Nomination.result was assigned to ENUM to make sure that the valid set of values was fixed and valid (Winner, Nominated, etc.), and this way data integrity was ensured.

Attributes that are essential to the identity of an entity (i.e. festival name, title, name of a person) had NOT NULL constraints.

UNIQUE constraints eliminate logical duplication, e.g. the identical festival in the identical year (ux_festival_name_year), or the identical person in the identical primary role (ux_person_name_role).

ER DIAGRAM



Entities:

Country, Festival, Award, Film, Person, Genre, Nomination.

Relationships:

Festival (1) is held in (1) Country.

Festival (1) presents (M) Awards.

Film (M) is produced in (1)Country.

Person (M) has (1) Country as nationality.

Film (M) has (M) person which is resolved by Film_person as associative entity.

Film (M) has (M) Genre.

Award (1) has (M) Nominations.

Film (M) recieves (M) Nominations which is resolved by the nomination associative entity.

Nomination (M) is associated with (M) Person which is resolved by the nomination_person associative entity.

Relational Mapping

Country (country_id, country_name)

Festival(festival_id, festival_name, year, city, country_id, start_date, end_date)

Film (film_id, title, release_year, language, country_id, runtime_minutes)

Person (person_id, name, primary_role, nationality_id)

Genre (genre_id, genre_name)

Award (Award_id, Award_name, year, category, festival_id)

Nomination (nomination_id, nomination_date, result, film_id, award_id, notes)

Film_person (film_person_id, film_id, person_id, role_in_film, credit_order)

Film_Genre (film_id, genre_id)

Nomination_Person (nomination_person_id, nomination_id, person_id, rol_in_nomination)

Normalization

The schema, as represented in the tables.sql, is in 3NF (Third normal Form). This is guaranteed in the design process of mapping off the ER model.

1NF: All the attributes are atomic (e.g. genre name is not a list and it is processed through Film Genre Table).

2NF: The schema has been converted to 1NF and all non-prime attributes are totally functionally dependent on the primary keys. Partial dependencies do not exist.

3NF: No transitive dependencies. Attributes that are not keys are fully reliant on the key. As in the example of the Festival table city and country id are dependent on the primary key of festival and not on each other.

Justification of not using certain ER concepts:

Weak Entity

Nomination might be represented as weak, but it is easier, more flexible and easy to query, update and reference in Nomination_Person using a regular entity that has the nomination_id. This scenario did not require weak entities.

Subtype / Supertype

Even though individuals have been assigned various positions (Director, Actor, Cinematographer), they do not need special qualities on each subtype. One Person table + Film_Person (role in film) will be easier and will meet the requirements completely. Adding subtypes would not be of any practical use.

Composite Attributes

Atomic fields are needed in relational databases. Such attributes as festival location or dates have already been broken into atomic columns (city, start date, end date). Composite attributes are not actually in tables but are merely conceptual.

Multivalued Attributes

Films have multiple genres. We decided to normalize and create two tables Genre and FilmGenre instead of violating 1NF by storing the various genres in one column. This facilitates queries, indexing and upholds 1NF and 3NF.

Derived Attributes

Derived values could be calculated in SQL query or views like film age. Keeping them would be redundant and go against 3NF. As such, derived attributes are not included in the

schema. 3NF Summary The schema is on Third Normal Form due to the reason that: all attributes are atomic (1NF), no half dependencies on compound keys (2NF), no transitive dependencies; all attributes are dependent on the primary key of their own table (3NF).

Assumptions

1. The simplicity of a film is in the fact that it has a single country of production. Multinational co-productions are out of the scope at the moment.
2. An individual possesses a primary nationality.
3. The primary role of an individual will be the most well-known role of the individual but they can as well be given credit in other roles in the individual films through the Film Person table.
4. Nomination table mostly captures the nominations at film level. The association with individual individuals is treated with the Nomination Person table which is flexible enough to assign wins to directors, actors etc.
5. The festivals issue publications that are named and dated.

Implementation

The SQL Data Definition Language (DDL) was used in the implementation of the database through the table.sql scripts. The script shows that it is professional practice by:

Setting up of the database Mashood 21978586 using Unicode character set to support the international requirement.

The process of temporarily turning off foreign key checks so that it is possible to create and drop tables flexibly.

The logical creation of tables that consider the foreign key dependencies (e.g. Country then Festival and Film).

Application of all the planned restrictions: primary keys, foreign keys, UNIQUE constraints, NOT NULL, and ENUM.

Enforcement of referential integrity by use of the InnoDB engine. Defining strategic indexes (e.g. idx_festival_year, idx_film_title) to make queries run faster.

Evidence: With a successful run of the tables.sql script in a MySQL client (e.g., command line or MySQL Workbench), the database and all of its tables will be created without error as displayed by the CREATE TABLE success messages.

Sample Data

The data used to fill in the database was complete and realistic, with data on two large international movie festivals (Berlin and Venice) in several years. The data that has been inserted is shown in a summarized form below:

Country Data:

It was represented by 46 countries including big film-producing countries. The major ones are: United States, France, Germany, Italy, United Kingdom, Japan, South Korea. Multicultural representation in Europe, Asia, Americas and Africa.

Festival Data:

13 editions of the festival in two large festivals: Berlin International Film Festival (2016-2023, no years 2021) Venice Film Festival (2017-2023 excluding 2020)

Award Structure:

20 different awards in both festivals. Berlin: Golden Bear, Silver Bear (Best Movie, Director, Performance, Screenplay) Venice: Golden Lion, Grand Jury Prize, Volpi Cups (Best Actor/Actress), Silver Lion.

Film Collection:

30 films from 2017-2023 Various languages: English, German, Spanish, French, Japanese, Korean, Persian, and so on.

Duration of operation: 66 minutes to 480 minutes.

Countries of production: the United States (7 films), Germany (3), France (2), Italy (2) and so on.

People Database:

The film industry includes 77 people. 30 Directors among which Yorgos Lanthimos, Jane Campion, Guillermo del Toro. 31 Stars such as Emma stone, Willem Dafoe, Colin Farrel, Penelope Cruz. Cinematographers 16 with Robbie Ryan, Yorick Le Saux. Different nationalities were represented.

Genre Classification:

Drama was the most widespread genre. Movies usually possess 1-3 genres. Examples: poor things = Comedy, Romance, Science Fiction. The Banshees of Inisherin = Tragedy, Comedy.

Nominations and Awards:

20 successful nominations achieved. Several awards of "Poor Things" (Golden Lion, Best Actor, Best Actress at Venice 2023). Winner of Golden bear (2022) and silver bear: Best Director (2023) is "Alcarras. End to end candidate monitoring and persons.

Relationships Mapped:

21 relations of film-persons (directors, actors, cinematographers).

50 film-genre classifications

26 associations of nomination-persons (connecting winners with a given individual).

Use of the Database

Design and implementation of Queries:

An elaborate code of SQL queries was developed and executed to get valuable information out of the film festival database. These questions show expertise in the use of basic and advanced SQL features, answering realistic information requirements.

Queries

Simple queries: basic queries, SELECT with conditions, sorting and functions.

1) Query: List all films released in 2022

The query is used as a basic filter to select new movies in the database. It is handy to create the current film lists or put the analysis into a particular year.

Snippet:

```
SELECT film_id, title, release_year, language
FROM Film
WHERE release_year = 2022
ORDER BY title;
```

output:

2) Query: List films longer than 120 minutes

Such query can be used to filter films by a numerical value (runtime), e.g., to find feature-length films or films with long duration to use in a programming analysis.

snippet:

```sql
SELECT title, release_year, runtime_minutes
FROM Film
WHERE runtime_minutes > 120
ORDER BY runtime_minutes DESC;
```

output:

title	release_year	runtime_minutes
The Works and Days	2020	480
There Is No Evil	2020	150
Poor Things	2023	141
By the Grace of God	2019	137
Robe of Gems	2022	136
Roma	2018	135
The Hand of God	2021	130
Spoor	2017	128
The Power of the Dog	2021	126
Touch Me Not	2018	125
Synonyms	2019	123
Félicité	2017	123
The Shape of Water	2017	123
Joker	2019	122
Wheel of Fortune and Fantasy	2021	121

15 rows in set (0.00 sec)

3) Query: Count films by country of origin (Top 10)

This analytical question looks at where the films in the database are from. This allows to see the most represented cinemas of different countries which is useful for cultural analysis and further insights.

Snippet:

```sql
SELECT c.country_name, COUNT(*) AS films_count
FROM Film f
JOIN Country c ON f.country_id = c.country_id
GROUP BY c.country_name
ORDER BY films_count DESC
LIMIT 10;
```

Output:

country_name	films_count
United States	6
France	2
Italy	2
Germany	2
Japan	2
Mexico	2
Poland	2
South Korea	1
Spain	1
Iran	1

10 rows in set (0.02 sec)

Advanced queries: complicated queries using JOINs, GROUP BY, aggregation and complex operations.

1) Count films by country of origin (top 10)

This analytical question gives a geographical summary of the film collection in the database. It assists in determining the most represented countries in terms of the cinema, and it is useful in the analysis of culture and festival.

Snippet:

Output:

```
+---------------+------------+
| country_name  | awards_won |
+---------------+------------+
| United States |          7 |
| France        |          2 |
| Ireland       |          2 |
| Mexico        |          2 |
| Spain         |          2 |
| Austria       |          1 |
| Germany       |          1 |
| Iran          |          1 |
| Italy         |          1 |
| New Zealand   |          1 |
+---------------+------------+
10 rows in set (0.06 sec)
```

2) Winners for a specific festival edition (Venice 2023)

This is a primary business intelligence inquiry that offers a full list of winners for the edition of the festival. It shows the ability to work with many relationships in the database to display combined data.

Snippet:

```sql
SELECT
  fest.festival_name,
  fest.year AS festival_year,
  aw.award_name,
  f.title AS film_title,
  GROUP_CONCAT(DISTINCT p.name SEPARATOR ', ') AS persons,
  n.result,
  n.nomination_date
FROM Award aw
JOIN Festival fest ON aw.festival_id = fest.festival_id
JOIN Nomination n ON n.award_id = aw.award_id AND n.result = 'Winner'
JOIN Film f ON n.film_id = f.film_id
LEFT JOIN Nomination_Person np ON np.nomination_id = n.nomination_id
LEFT JOIN Person p ON np.person_id = p.person_id
WHERE fest.festival_name = 'Venice Film Festival' AND fest.year = 2023
GROUP BY aw.award_id, n.nomination_id
ORDER BY aw.award_name;
```

Output:

festival_name	festival_year	award_name
Venice Film Festival	2023	Golden Lion
Venice Film Festival	2023	Volpi Cup for Best Actor
Venice Film Festival	2023	Volpi Cup for Best Actress

film_title	persons	result	nomination_date
Poor Things	Emma Stone, Willem Dafoe, Yorgos Lanthimos	Winner	2023-09-09
Poor Things	Emma Stone, Willem Dafoe, Yorgos Lanthimos	Winner	2023-09-09
Poor Things	Emma Stone, Willem Dafoe, Yorgos Lanthimos	Winner	2023-09-09

3) Films with their genres.

This query shows which movies are associated with which genres in an easy-to-use way. Using GROUP_CONCAT, we can convert a complicated many-to-many relationship into something useful.

Snippet:

```
SELECT f.film_id, f.title, f.release_year,
       IFNULL(GROUP_CONCAT(DISTINCT g.genre_name ORDER BY g.genre_name SEPARATOR ', '),
'Unknown') AS genres
FROM Film f
LEFT JOIN Film_Genre fg ON f.film_id = fg.film_id
LEFT JOIN Genre g ON fg.genre_id = g.genre_id
GROUP BY f.film_id, f.title, f.release_year
ORDER BY f.release_year DESC, f.title;
```

Output:

film_id	title	release_year	genres
31	My New Film	2025	Unknown
20	Poor Things	2023	Comedy, Romance, Science Fiction
4	A E I O U - A Quick Alphabet of Love	2022	Drama, Romance
3	Alcarràs	2022	Drama
21	All the Beauty and the Bloodshed	2022	Documentary
22	L'immensità	2022	Drama
2	Robe of Gems	2022	Drama, Thriller
1	Sparta	2022	Drama
23	The Banshees of Inisherin	2022	Drama, Tragicomedy
5	Bad Luck Banging or Loony Porn	2021	Comedy, Drama, Satire
24	Happening	2021	Drama
7	Introduction	2021	Drama
26	The Hand of God	2021	Drama
25	The Power of the Dog	2021	Drama, Western
6	Wheel of Fortune and Fantasy	2021	Anthology, Drama, Romance
9	Never Rarely Sometimes Always	2020	Drama
27	Nomadland	2020	Drama
10	The Works and Days	2020	Drama
8	There Is No Evil	2020	Anthology, Drama
12	By the Grace of God	2019	Drama
28	Joker	2019	Crime, Drama, Psychological thriller
11	Synonyms	2019	Drama
13	System Crasher	2019	Drama
15	Mug	2018	Black comedy, Drama
29	Roma	2018	Drama
16	The Heiresses	2018	Drama
14	Touch Me Not	2018	Documentary, Drama
18	Félicité	2017	Drama, Musical
17	On Body and Soul	2017	Drama, Romance
19	Spoor	2017	Drama, Eco-thriller, Thriller
30	The Shape of Water	2017	Drama, Romantic fantasy

31 rows in set (0.33 sec)

Advanced Concepts (views, procedures)

Views:

1) Vw_film_genres

This view brings the complex genre concatenation query into physical existence. It allows applications and users to access movie genre information without being aware of the underlying tables.

Snippet:

```
SELECT f.film_id, f.title, f.release_year,
       IFNULL(GROUP_CONCAT(DISTINCT g.genre_name ORDER BY g.genre_name SEPARATOR ', '),
'Unknown') AS genres
FROM Film f
LEFT JOIN Film_Genre fg ON f.film_id = fg.film_id
LEFT JOIN Genre g ON fg.genre_id = g.genre_id
GROUP BY f.film_id, f.title, f.release_year
ORDER BY f.release_year DESC, f.title;


SELECT * FROM vw_film_genres WHERE title LIKE '%Poor Things%';
```

output:

```
+---------+-------------+--------------+--------------------------------+----------+-----------------+------------+
| film_id | title       | release_year | genres                         | language | runtime_minutes | country_id |
+---------+-------------+--------------+--------------------------------+----------+-----------------+------------+
|      20 | Poor Things |         2023 | Comedy, Romance, Science Fiction | English  |             141 |          4 |
+---------+-------------+--------------+--------------------------------+----------+-----------------+------------+
1 row in set (0.03 sec)
```

2) Vw_winners_summary

This overall view acts as a reporting device that brings together all winning information from all festivals. It is especially useful for journalists, researchers and festival organisers looking for quick access to winner summaries.

Snippet:

```
DROP VIEW IF EXISTS vw_winners_summary;
CREATE VIEW vw_winners_summary AS
SELECT
  fest.festival_id,
  fest.festival_name,
  fest.year AS festival_year,
  aw.award_id,
  aw.award_name,
  f.film_id,
  f.title AS film_title,
  GROUP_CONCAT(DISTINCT p.name ORDER BY p.name SEPARATOR ', ') AS persons,
  n.nomination_date,
  n.result
FROM Award aw
JOIN Festival fest ON aw.festival_id = fest.festival_id
JOIN Nomination n ON n.award_id = aw.award_id
JOIN Film f ON n.film_id = f.film_id
LEFT JOIN Nomination_Person np ON n.nomination_id = np.nomination_id
LEFT JOIN Person p ON np.person_id = p.person_id
WHERE n.result = 'Winner'
GROUP BY fest.festival_id, aw.award_id, n.nomination_id;


SELECT festival_name, festival_year, award_name, film_title, persons
FROM vw_winners_summary
WHERE festival_name='Venice Film Festival' AND festival_year=2023;
```

Output:

festival_name	festival_year	award_name	film_title	persons
Venice Film Festival	2023	Golden Lion	Poor Things	Emma Stone, Willem Dafoe, Yorgos Lanthimos
Venice Film Festival	2023	Volpi Cup for Best Actress	Poor Things	Emma Stone, Willem Dafoe, Yorgos Lanthimos
Venice Film Festival	2023	Volpi Cup for Best Actor	Poor Things	Emma Stone, Willem Dafoe, Yorgos Lanthimos

3 rows in set (0.04 sec)

Stored Procedure:

1) Sp_add_country

This process includes proper error management and transaction management logic for country insertion. It helps in maintaining the data integrity and facilitates an easy and safe interface to add new countries.

Snippet:

```sql
DROP PROCEDURE IF EXISTS sp_add_country;
DELIMITER $$

CREATE PROCEDURE sp_add_country (
    IN p_country_name VARCHAR(120)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error inserting country.';
    END;

    START TRANSACTION;

    INSERT INTO Country (country_name)
    VALUES (p_country_name);

    COMMIT;

    SELECT CONCAT('Inserted country: ', p_country_name) AS message;
END $$

DELIMITER ;



CALL sp_add_country('kuwait');
```

Output:

```
+-------------------------+
| message                 |
+-------------------------+
| Inserted country: kuwait |
+-------------------------+
1 row in set (0.44 sec)
```

2) Sp_top_awarded_films

You can use this procedure to find the most awarded film in a year of your choice. This shows how to use input parameters to generate flexible, reusable database code.

Snippet:

```
DROP PROCEDURE IF EXISTS sp_top_awarded_films;
DELIMITER $$
CREATE PROCEDURE sp_top_awarded_films(
  IN p_year INT,        -- award year to consider (Award.year / Festival.year)
  IN p_top_n INT        -- how many top films to return
)
BEGIN
  /*
    Returns a resultset:
      film_id, title, release_year, wins_count
  */
  SELECT
    f.film_id,
    f.title,
    f.release_year,
    COUNT(*) AS wins_count
  FROM Nomination n
  JOIN Award a ON n.award_id = a.award_id
  JOIN Film f  ON n.film_id = f.film_id
  WHERE a.year = p_year
    AND n.result = 'Winner'
  GROUP BY f.film_id
  ORDER BY wins_count DESC, f.title
  LIMIT p_top_n;
END $$
DELIMITER ;


CALL sp_top_awarded_films(2023, 10);
```

Output:

film_id	title	release_year	wins_count
20	Poor Things	2023	3
4	A E I O U - A Quick Alphabet of Love	2022	1
3	Alcarràs	2022	1
2	Robe of Gems	2022	1
1	Sparta	2022	1

5 rows in set (0.04 sec)

Database Connectivity and python implementation

A full-fledged application in Python developed to show practical database connectivity and complete CRUD (Create, Read, Update, Delete) operations through programming interface. The application of the film festival database has a user-friendly menu system.

Implementation Architecture

1. Database Configuration & Security.

Created a dedicated database user ff_user@localhost with required privileges.

Utilized parameterized queries to prevent SQL injection.

Ensured right connection handling with error handling.

2. Modular Design.

Broke apart the details preventing database usability.

Organized the querying functions and different CRUD operations.

Implemented a main menu system for user interaction.

Implementation

1. Database connection setup

```
DB_CONFIG = {
    "host": "localhost",
    "user": "ff_user",
    "password": "ChooseAStrongPassword123!",
    "database": "Mashood_21978586",
    "auth_plugin": "mysql_native_password"
}


def get_conn():
    return mysql.connector.connect(**DB_CONFIG)
```

2. Query Execution Wrapper

```python
def run_query(sql, params=None, fetch=True):
    conn = get_conn()
    cur = conn.cursor()

    try:
        cur.execute(sql, params or ())
        if fetch:
            rows = cur.fetchall()
            columns = [desc[0] for desc in cur.description]
            print("\nRESULT:")
            print(columns)
            for r in rows:
                print(r)
            return rows
        else:
            conn.commit()
            return True

    except Error as e:
        print("ERROR:", e)
    finally:
        cur.close()
        conn.close()
```

Operation

For query: we'll be using this Query 4 as an example.

Query 4:

Snippet:

```
def query_4():
    sql = """
        SELECT
          fest.festival_name,
          fest.year AS festival_year,
          aw.award_name,
          f.title AS film_title,
          GROUP_CONCAT(DISTINCT p.name SEPARATOR ', ') AS persons,
          n.result,
          n.nomination_date
        FROM Award aw
        JOIN Festival fest ON aw.festival_id = fest.festival_id
        JOIN Nomination n ON n.award_id = aw.award_id AND n.result = 'Winner'
        JOIN Film f ON n.film_id = f.film_id
        LEFT JOIN Nomination_Person np ON np.nomination_id = n.nomination_id
        LEFT JOIN Person p ON np.person_id = p.person_id
        WHERE fest.festival_name = 'Venice Film Festival' AND fest.year = 2023
        GROUP BY aw.award_id, n.nomination_id
        ORDER BY aw.award_name;
    """
    run_query(sql)
```

Output:

```
RESULT:
['title', 'release_year', 'runtime_minutes']
('The Works and Days', 2020, 480)
('There Is No Evil', 2020, 150)
('Poor Things', 2023, 141)
('By the Grace of God', 2019, 137)
('Robe of Gems', 2022, 136)
('Roma', 2018, 135)
('The Hand of God', 2021, 130)
('Spoor', 2017, 128)
('The Power of the Dog', 2021, 126)
('Touch Me Not', 2018, 125)
('Synonyms', 2019, 123)
('Félicité', 2017, 123)
('The Shape of Water', 2017, 123)
('Joker', 2019, 122)
('Wheel of Fortune and Fantasy', 2021, 121)
```

Insertion:

```python
def insert_film():
    print("\n--- INSERT NEW FILM ---")
    title = input("Enter film title: ")
    year = input("Enter release year: ")
    lang = input("Enter language: ")
    runtime = input("Enter runtime in minutes: ")
    country_id = input("Enter country_id (1-35): ")

    sql = """
        INSERT INTO Film (title, release_year, language, runtime_minutes, country_id)
        VALUES (%s, %s, %s, %s, %s);
    """

    run_query(sql, (title, year, lang, runtime, country_id), fetch=False)
    print("Film inserted successfully!\n")
```

Update:

```python
def update_film():
    print("\n--- UPDATE FILM ---")
    film_id = input("Enter Film ID to update: ")

    print("""
    What do you want to update?
    1 – Title
    2 – Release Year
    3 – Language
    4 – Runtime
    """)
```

Deletion:

```python
def delete_film():
    print("\n--- DELETE FILM ---")
    show_all_films()

    film_id = input("\nEnter Film ID to delete: ")

    sql = "DELETE FROM Film WHERE film_id = %s"
    run_query(sql, (film_id,), fetch=False)

    print("Film deleted successfully!\n")
```

User Interface Menu

Snippet:

```python
def main_menu():
    while True:
        print("""
========= FILM FESTIVAL MENU =========
1. Show all films
2. Query 1 – Films from 2022
3. Query 2 – Top 10 countries by number of films
4. Query 3 – Films with runtime > 120 mins
5. Query 4 – Winners at Venice 2023
6. Query 5 – Top 10 countries by awards won
7. Query 6 – Films + Genres
8. Insert Film
9. Update Film
10. Delete Film
0. Exit
======================================
""")
```

Output:

```
========= FILM FESTIVAL MENU =========
1. Show all films
2. Query 1 – Films from 2022
3. Query 2 – Top 10 countries by number of films
4. Query 3 – Films with runtime > 120 mins
5. Query 4 – Winners at Venice 2023
6. Query 5 – Top 10 countries by awards won
7. Query 6 – Films + Genres
8. Insert Film
9. Update Film
10. Delete Film
0. Exit
======================================
```

Discussion

This project successfully resulted with a complete running, well designed and rich database of films. The main achievement was to develop a well-formed and robust schema to model a complex real world domain and to instantiate it through a careful process of implementation, testing and application integration.

Challenges Faced

1. The most serious challenge was the modelling of the complex many-to-many relationships between person, film, and nomination Choosing the Nomination_Person junction table was beneficial to avoid attributing awards to the correct people and not duplicating the nomination record.
2. While inserting data, it became clear that the depth of the tables relative to each other wasn't easy to populate. To construct the dataset in an orderly format without any errors, it became essential to use INSERT IGNORE and subqueries on foreign keys.
3. Throughout the assignment, there were complex decisions to make sure that the design is complex enough to show that we mastered the task (Nomination_Person) but also not too complex to be unmanageable.

Limitations and Future Improvements.

We can enhance the system with a Role lookup table instead of free-text role_in_film & primary_role to standardise role and eliminate typos.

Real application should have a user system with permissions levels. For example, admin one who will do data entry, and public user one who will do read-only queries.

More sophisticated analytics can also involve implementing more stored procedures or user-defined functions for assessing, say, a festival's "internationality index", by calculating the diversity of countries of origin of the winning films.

The logical next step is to construct a web-based front-end (for instance, using Flask or Django) that uses the Python connectivity to provide a user interface for all database operations. This will make the system usable by non-technical users.

Conclusion

This project has enabled me to understand the complete database development process, including conceptual ER design, normalization, physical implementation, advanced SQL programming, and application connectivity. The database is a solid starting point which, when enhanced with the above additions, could become a full film festival management and information system.

References

Wikipedia contributors. (2025a, September 7). *Berlin International Film Festival*. Wikipedia.

https://en.wikipedia.org/wiki/Berlin_International_Film_Festival

Wikipedia contributors. (2025b, November 18). *Venice Film Festival*. Wikipedia.

https://en.wikipedia.org/wiki/Venice_Film_Festival